TAREA M27 – CD-YAIR PALOMARES

- Extrae en línea la base de datos "Iris.csv" de la siguiente liga y agrégala a un DataFrame en Python.

```
import pandas as pd
import matplotlib.pyplot as mp
import seaborn as sns
import os
import warnings
warnings.filterwarnings("ignore")

os.chdir("C:/Users/yair_/OneDrive/Desktop/Ciencia de Datos/Módulo 7 - DataFrames")
data = pd.read_csv("iris.csv")
data
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica
149	5.9	3.0	5.1	1.8	Virginica

150 rows × 5 columns

- Agrupa las observaciones de la tabla descargada mediante el algoritmo de K-Medias sin realizar ningún tipo de transformación de los datos originales. Para dicho efecto, determina el número óptimo de clusters tanto por el criterio del gráfico de codo así como por el indicador Silhouette.

```
data = data.drop(columns="variety")
data
```

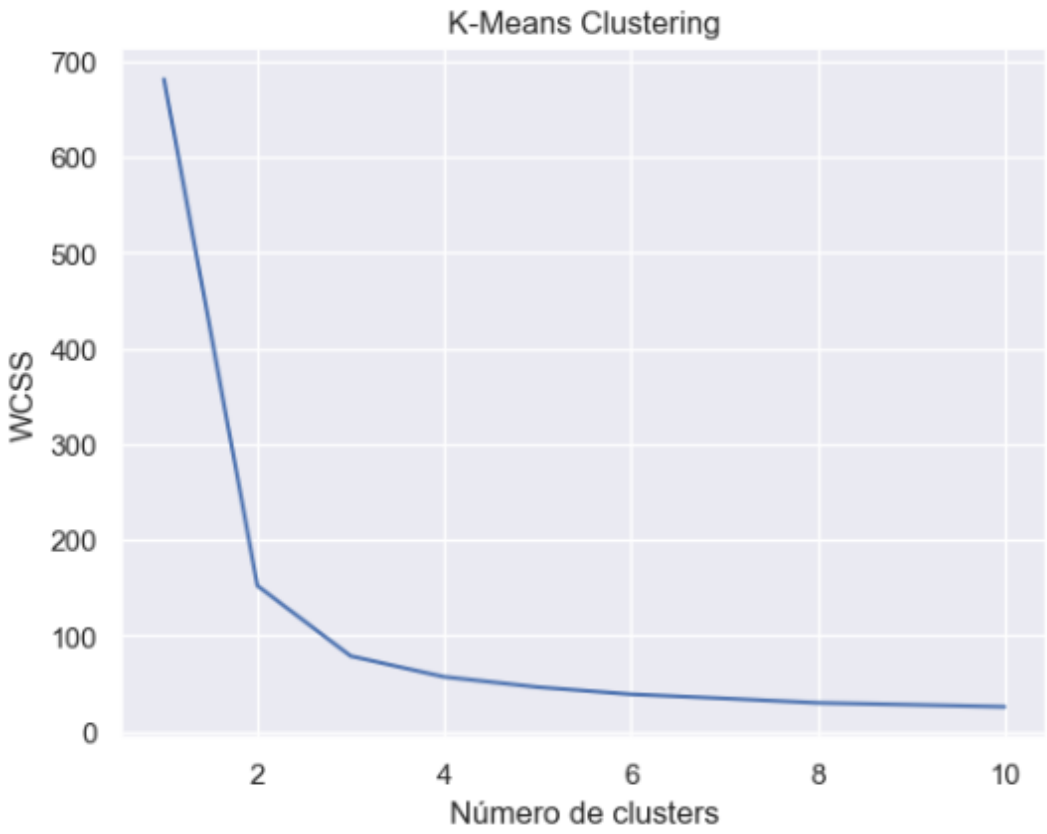
	sepal.length	sepal.width	petal.length	petal.width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

```
from sklearn.cluster import KMeans
wcss = []
for index in range(1,11):
    kmeans = KMeans(n_clusters = index, init="k-means++", random_state=2)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)
```

```
# Graficación del "Codo de Jambú"
sns.set()
mp.plot(range(1,11), wcss)
mp.title("K-Means Clustering")
mp.xlabel("Número de clusters")
mp.ylabel("WCSS") #Weighted cluster sum of squares
```

```
Text(0, 0.5, 'WCSS')
```




```
# Encontrar el número de grupos óptimo a través del "Criterio de Silueta (Silhouette)"
from sklearn import datasets
from sklearn.metrics import silhouette_score
```

```python
x = data.to_numpy() #Se convierte a un arreglo numpy porque así lo requiere este comando
for j in range(2,12):
    kmeans = KMeans(n_clusters = j, random_state=42)
    kmeans.fit_predict(x)
    #Calculamos el score de Silueta
    score = silhouette_score(x, kmeans.labels_, metric="euclidean")
    print("Score Silhouette;", "k = ", j, ":", score)
```

```
Score Silhouette; k =  2 : 0.6810461692117467
Score Silhouette; k =  3 : 0.5528190123564101
Score Silhouette; k =  4 : 0.49805050499728803
Score Silhouette; k =  5 : 0.4887488870931063
Score Silhouette; k =  6 : 0.36483400396700355
Score Silhouette; k =  7 : 0.34974816211612175
Score Silhouette; k =  8 : 0.3574536925852726
Score Silhouette; k =  9 : 0.339448863639371
Score Silhouette; k =  10 : 0.2973144972185989
Score Silhouette; k =  11 : 0.318481930245191
```

```python
kmeans = KMeans(n_clusters = 2, init="k-means++", random_state=42)
cluster_values = kmeans.fit_predict(data)
print(cluster_values)
```

```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]
```

- Repite el ejercicio anterior después de haber transformado tus datos originales a dos columnas, mediante reducción por PCA. Gráfica los grupos obtenidos considerando las dos dimensiones resultantes.

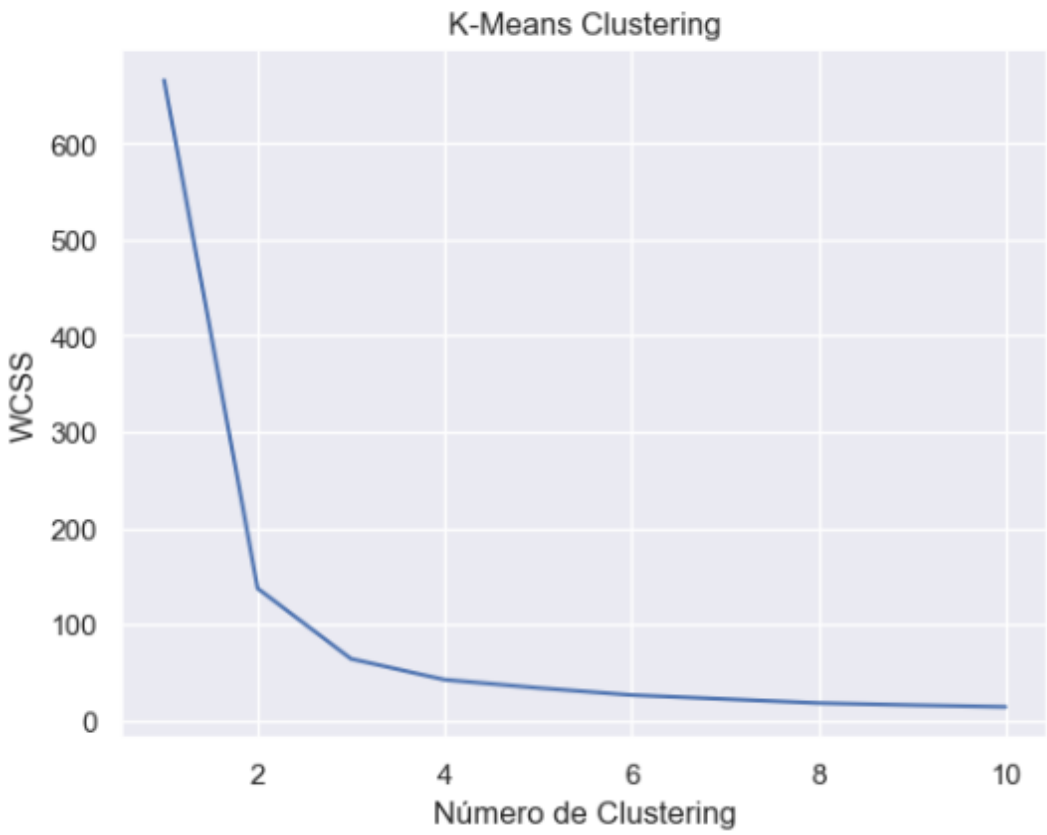
```python
# Reducción de dimensiones por PCA
from sklearn import decomposition
pca = decomposition.PCA(n_components=2)
pca.fit(x)
x = pca.transform(x)
x
```

```
array([[-2.68412563,  0.31939725],
       [-2.71414169, -0.17700123],
       [-2.88899057, -0.14494943],
       [-2.74534286,  0.31829898],
```

```python
from sklearn.cluster import KMeans
wcss2 = []
for index in range(1,11):
    kmeans = KMeans(n_clusters = index, init="k-means++", random_state=2)
    kmeans.fit(x)
    wcss2.append(kmeans.inertia_)
```

```python
# Graficación del "Codo de Jambú"
sns.set()
mp.plot(range(1,11), wcss2)
mp.title("K-Means Clustering")
mp.xlabel("Número de Clustering")
mp.ylabel("WCSS") #Weighted cluster sum of squares
```

```
Text(0, 0.5, 'WCSS')
```



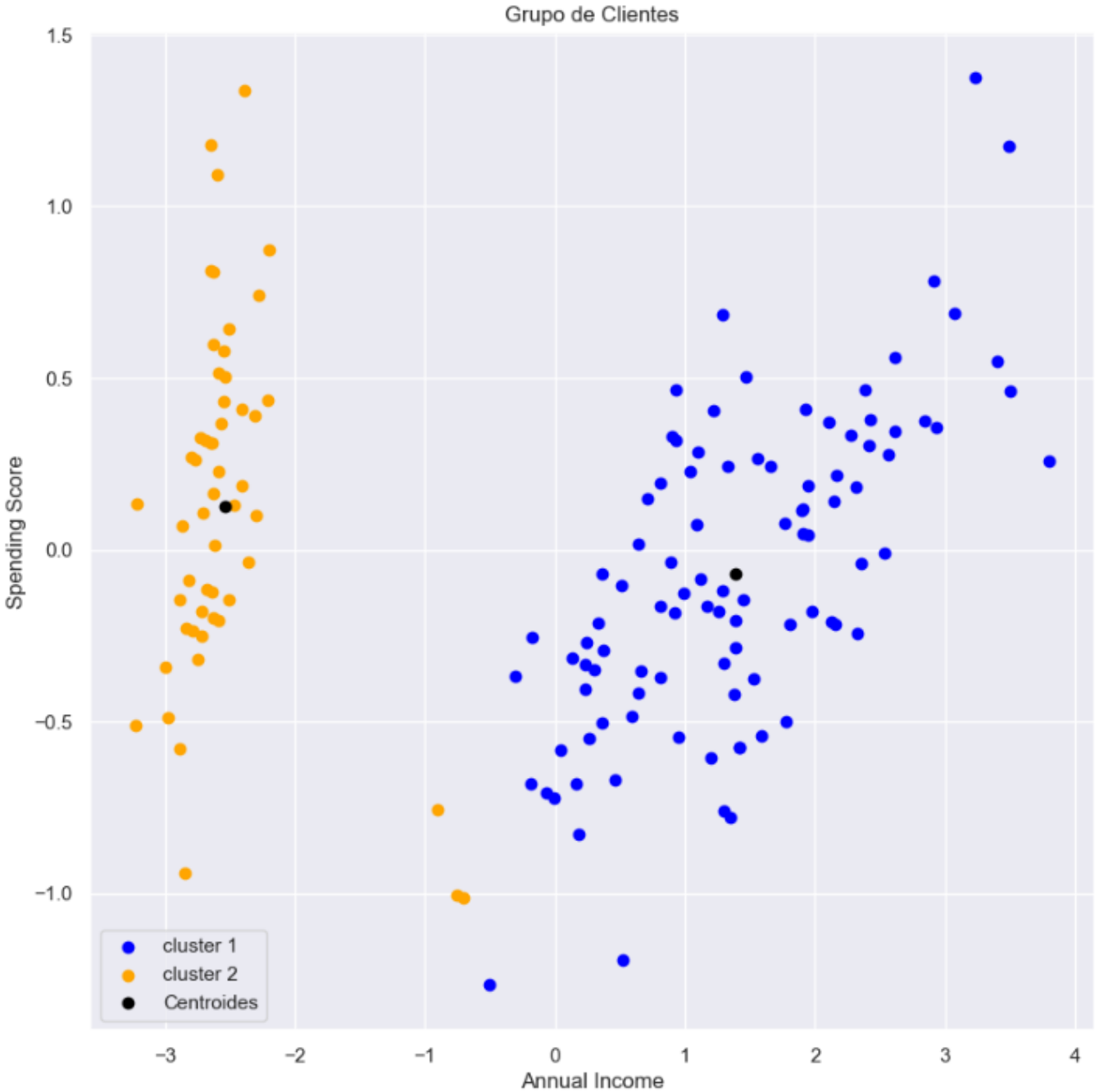
```python
kmeans = KMeans(n_clusters = 2, init="k-means++", random_state=42)
cluster_values_PCA = kmeans.fit_predict(x)
print(cluster_values_PCA)
```

```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]
```

```python
mp.figure(figsize=(10, 10))
mp.scatter(x[cluster_values_PCA==0, 0], x[cluster_values_PCA==0, 1], c="blue", label="cluster 1")
mp.scatter(x[cluster_values_PCA==1, 0], x[cluster_values_PCA==1, 1], c="orange", label="cluster 2")

mp.scatter(kmeans.cluster_centers_[:,0], kmeans.cluster_centers_[:,1], c="black", label="Centroides")
mp.title("Grupo de Clientes")
mp.xlabel("Annual Income")
mp.ylabel("Spending Score")
mp.legend()
```

```
<matplotlib.legend.Legend at 0x1ed1f338090>
```



- Compara los resultados de los incisos 2) y 3). ¿Obtuviste los mismos resultados de agrupamiento? Comenta sobre la ventaja práctica de haber efectuado la transformación por PCA antes de aplicar el algoritmo K-Medias.

```
# Comparación de los dos resultados

if (cluster_values == cluster_values_PCA).all():
    print("Si se obtuvieron los mismos resultados de agrupamiento")
else:
    print("No se obtuvieron los mismos resultados de agrupamiento")

print("La ventaja de utilizar el método PCA al momento de agrupar por k-medias, es resumir los datos multivariables y lograr obtener una mejor visualiza
```

```
Si se obtuvieron los mismos resultados de agrupamiento
La ventaja de utilizar el método PCA al momento de agrupar por k-medias, es resumir los datos multivariables y lograr obtener una mejor visualización de
los resultados del agrupamiento.
```